Manual técnico — Vivent académico

1. Requisitos del sistema

Componente	Versión
Docker Desktop	reciente (motor Linux/WSL2)
Java JDK	25 LTS (solo para ejecutar backend fuera de contenedor y pruebas)
Node.js	24 LTS
pnpm	11.1.3 (vía Corepack)
PostgreSQL	16/17 (contenedor)
Navegadores	Chromium/Firefox/Safari recientes

2. Instalación y ejecución local

Resumen (detalle paso a paso en `INSTRUCCIONES_LOCAL.md` de la raíz):

```
 git clone https://github.com/wilfero/vivent-academico.git
cd vivent-academico
# Modo A (todo Docker)
cd docker && docker compose -f compose.local.yml up -d --build
# Modo B (desarrollo)
cd docker && docker compose -f compose.db.yml up -d
cd ../backend && ./mvnw spring-boot:run
cd ../frontend && pnpm install && pnpm dev
```

Bootstrap de operaciones demo (tras levantar el stack):

```
python database/bootstrap_demo.py
```

3. Configuración

Variables de entorno (ejemplo sin credenciales en `.env.example`):

Variable	Uso
`VIVENT_DB_URL`	JDBC de PostgreSQL (`jdbc:postgresql://localhost:5433/vivent`)
`VIVENT_DB_USERNAME` / `VIVENT_DB_PASSWORD`	credenciales de la BD
`VIVENT_JWT_ACCESS_TOKEN_TTL`	TTL del access token (por defecto `PT30M`)
`VIVENT_REFRESH_TOKEN_TTL`	TTL del refresh token (por defecto `PT168H` = 7 días)
`VIVENT_JWT_SECRET` / `VIVENT_JWT_ISSUER`	firma y emisor del JWT
`NEXT_PUBLIC_API_BASE_URL`	base de la API para el frontend
`NEXT_PUBLIC_DEFAULT_TENANT`	tenant de demo
`VIVENT_INTEGRATIONS_OPENFOODFACTS_BASE_URL`	base de la API externa Open Food Facts
`VIVENT_PLATFORM_*`	panel master SaaS (opcional en local)

4. Estructura

```
backend/src/main/java/com/corpoviva/vivent/
  auth/ tenant/ companies/ branches/ users/ permissions/ subscriptions/
  products/ inventory/ purchases/ sales/ transfers/ adjustments/ closures/
  reports/ audit/ barcodes/ storefront/ conversations/ email/ platform/ common/ security/
frontend/src/
  app/ components/ features/ lib/ messages/
database/migrations/ database/seed/
docker/ docs/ postman/
```

5. Autenticación y sesiones (access + refresh token)

```
POST /api/v1/auth/login    { tenantSlug, username, password }
  → 200 ApiResponse{ accessToken (JWT, 30 min), refreshToken (opaco, 7 días), permissions...
}

POST /api/v1/auth/refresh  { refreshToken }
  → 200 con NUEVO accessToken + NUEVO refreshToken (rotación; el presentado queda revocado)
  → 401 INVALID_TOKEN si está revocado/expirado

POST /api/v1/auth/logout   { refreshToken }
  → revoca el refresh y registra auditoría LOGOUT
```

Reglas de seguridad:

- El refresh token se almacena hasheado con SHA-256 (`refresh_tokens.token_hash`, único); el token crudo se muestra una sola vez al cliente.

- Cambiar/resetear la contraseña revoca todos los refresh tokens del usuario e incrementa `auth_version` (invalida también access tokens previos).
- El frontend renueva proactivamente 1 minuto antes de expirar y de forma reactiva ante cualquier 401 `INVALID_TOKEN` (una sola renovación en vuelo compartida por solicitudes concurrentes).
- Contraseñas con BCrypt; sin tokens ni secretos en logs.

6. Base de datos y Docker

- El esquema se crea desde cero con migraciones Flyway versionadas (`database/migrations`).
- Los datos de prueba cargan con seeds idempotentes (`database/seed`): catálogos con ≥5 registros y operación demo (empresa `demo`).
- `docker/compose.db.yml` levanta solo la BD; `docker/compose.local.yml` levanta BD + backend + frontend.
- Regla de versionado: nunca editar migraciones ya aplicadas; archivos nuevos con timestamp UTC de 14 dígitos; `out-of-order=true` .

7. APIs externas (por integrante)

Documentación completa de las tres integraciones con ejemplos de éxito y error: ver `APIS_EXTERNAS.md` .

8. Pruebas

```
cd backend && ./mvnw test          # 25 pruebas de integración (Testcontainers + PostGIS)
cd frontend && pnpm exec tsc --noEmit && NEXT_PRIVATE_BUILD_WORKER=1 pnpm run build
```

Evidencias y casos: ver `evidencias/EVIDENCIAS_PRUEBAS.md` .

9. Publicación en la nube

Guía paso a paso (Neon + Render + Vercel) con variables y checklist de evidencia: ver `PUBLICACION_CLOUD.md` .